Student Record and Academic Management System

Data Organization using Python

Student Name: Nikita Dnyaneshwar Udavant

PRN Number: 2127UDSF1031

1. Objective

To make a simple Student Record Management System using Python. It stores student details and helps to add, search, update, delete and display records. It can also calculate average marks and show student details by department.

2. Problem Analysis

The system stores roll number, name, department, subjects, marks and attendance. The user enters details or chooses an operation. The program displays the requested information or result.

3. Data Structures Used

Structure	Use
String	Name, department and contact details
List	Student records, subjects and marks
Tuple	Fixed details such as registration number or date of birth
Set	Unique departments or subjects
Dictionary	Details of one student

4. Algorithms

4.1 Add Student

1. Start
2. Enter roll number, name and department
3. Enter subjects and marks
4. Store details
5. Display success message
6. Stop

4.2 Search Student

1. Start
2. Enter roll number
3. Search student records
4. If found, display details
5. Otherwise display 'Student not found'
6. Stop

4.3 Update Record

1. Start
2. Enter roll number
3. Search for student
4. If found, enter and save new details
5. Otherwise display 'Student not found'
6. Stop

4.4 Delete Record

1. Start
2. Enter roll number
3. Search for student
4. If found, remove record

5. Otherwise display 'Student not found'

6. Stop

4.5 Display Records

1. Start

2. Check student list

3. If list is empty, display 'No records'

4. Otherwise display all records

5. Stop

4.6 Calculate Average

1. Start

2. Enter roll number

3. Find student record

4. Add marks and divide by number of marks

5. Display average

6. Stop

4.7 Find Highest Scorer

1. Start

2. Calculate average for each student

3. Compare averages

4. Keep student with highest average

5. Display student's name

6. Stop

4.8 List by Department

1. Start

2. Enter department name

3. Check each student

4. Display students from that department

5. Stop

4.9 Count Students

1. Start

2. Count student records

3. Display total number

4. Stop

4.10 Generate Report

1. Start

2. Display total student count

3. Show highest scorer if records exist

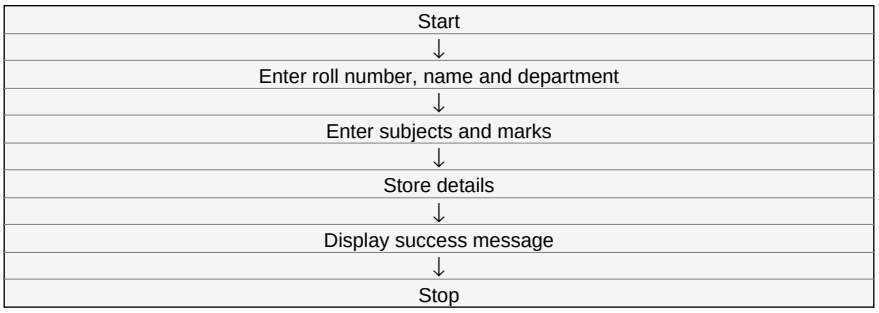
4. Display report

5. Stop

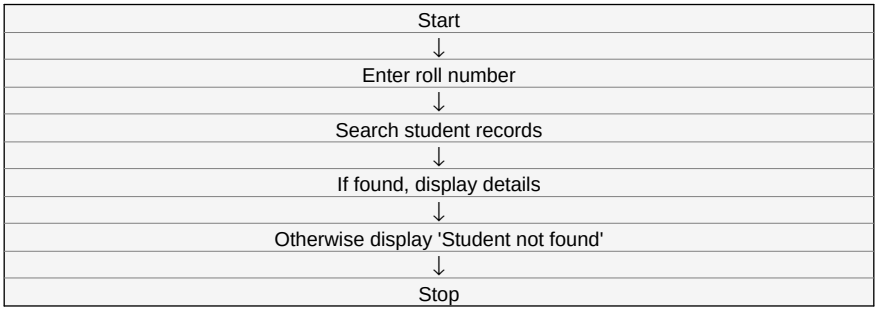
5. Flowcharts

Flowcharts show the steps in order. Each box is connected with an arrow.

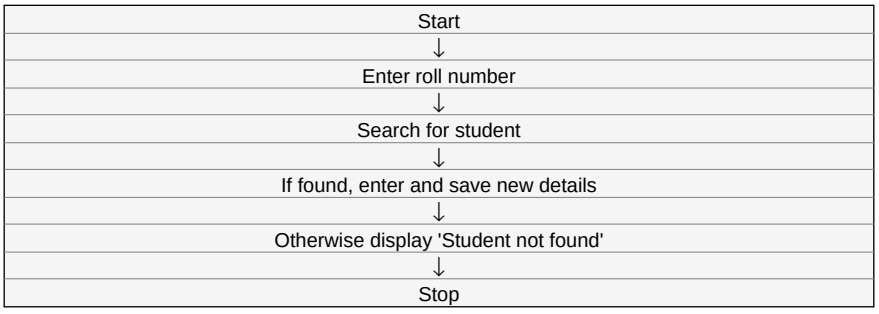
Add Student



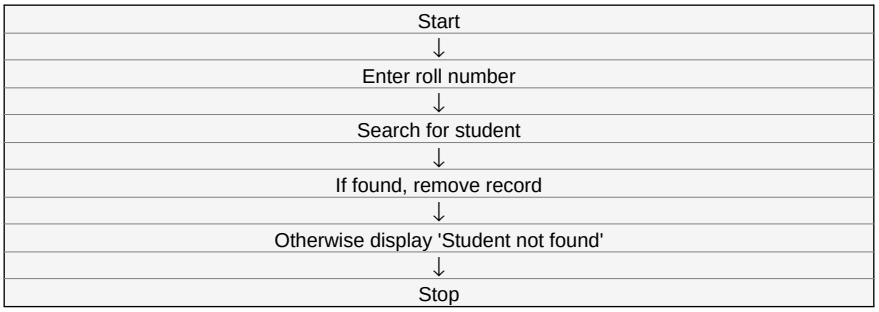
Search Student



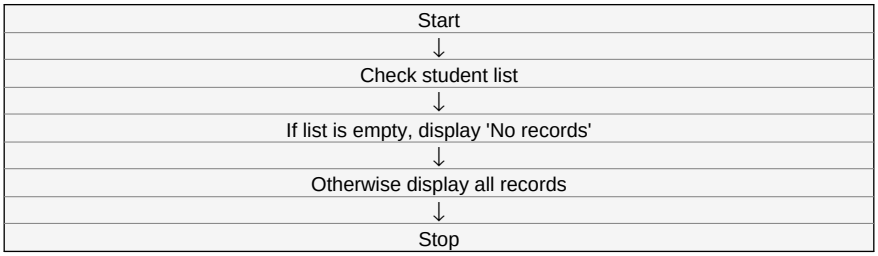
Update Record



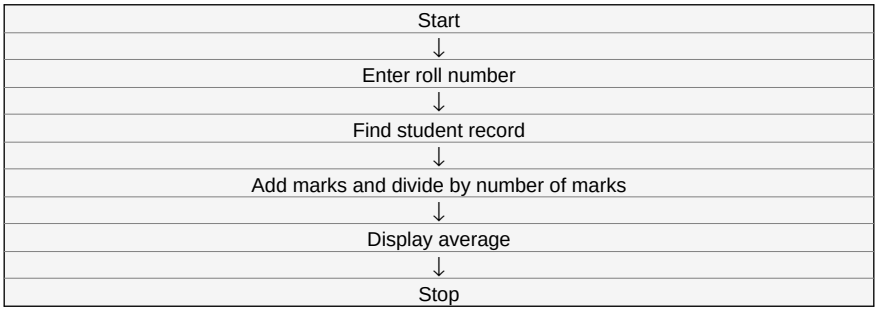
Delete Record



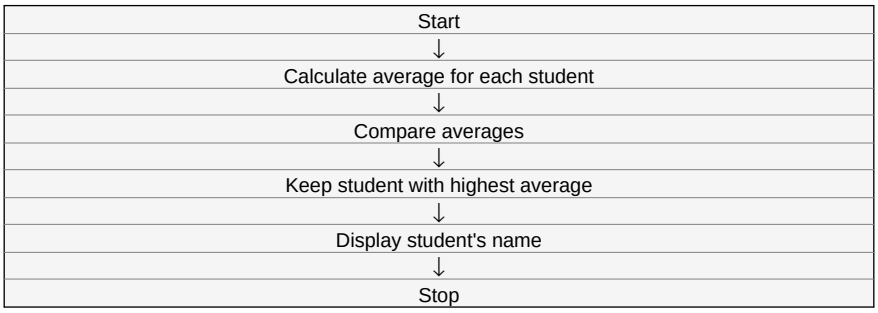
Display Records



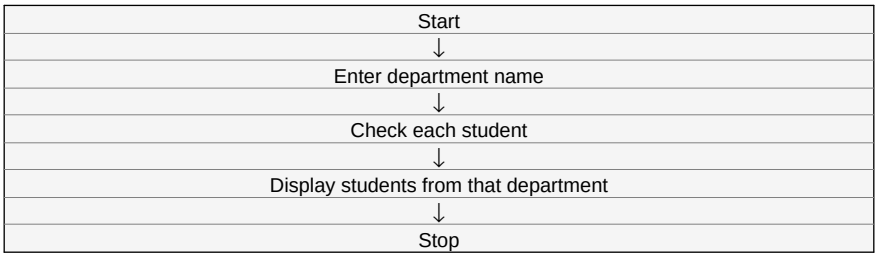
Calculate Average



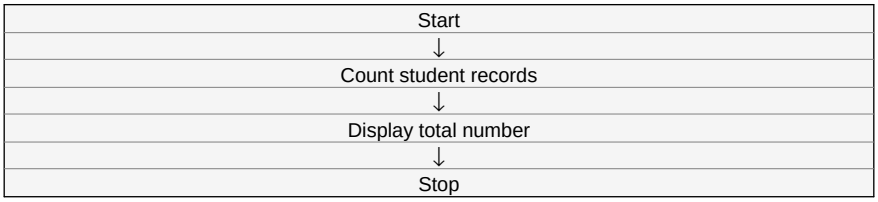
Find Highest Scorer



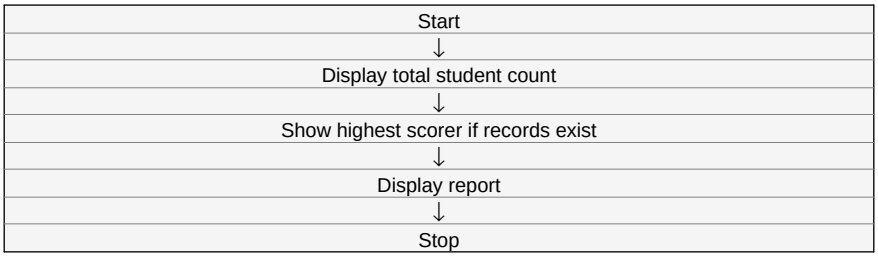
List by Department



Count Students



Generate Report



6. Pseudocode

Add Student

```
START
INPUT student details
STORE details in student list
DISPLAY success message
STOP
```

Search Student

```
START
INPUT roll number
SET found = false
CHECK each student
IF roll number matches, DISPLAY record
IF not found, DISPLAY message
STOP
```

Update Record

```
START
INPUT roll number
SEARCH record
IF found, INPUT new details and UPDATE
ELSE DISPLAY not found
STOP
```

Delete Record

```
START
INPUT roll number
SEARCH record
IF found, DELETE record
ELSE DISPLAY not found
STOP
```

Calculate Average

```
START
INPUT marks
TOTAL = sum of marks
AVERAGE = TOTAL / number of marks
DISPLAY AVERAGE
STOP
```

Find Highest Scorer

```
START
COMPARE student averages
KEEP highest average
DISPLAY student name
STOP
```

7. Python Source Code

This is a simple Python 3 program using lists, dictionaries, loops, functions and basic if statements.

```
students = []

def find_student(roll):
    for student in students:
        if student["roll"] == roll:
            return student
    return None

def add_student():
    roll = input("Roll number: ")
    if find_student(roll) is not None:
        print("This roll number already exists.")
        return

    name = input("Name: ")
    department = input("Department: ")
    subjects = input("Subjects separated by commas: ").split(",")
    marks_text = input("Marks separated by commas: ").split(",")

    marks = []
    for mark in marks_text:
        marks.append(float(mark.strip()))

    student = {
        "roll": roll,
        "name": name,
        "department": department,
        "subjects": subjects,
        "marks": marks
    }
    students.append(student)
    print("Student added.")

def search_student():
    roll = input("Enter roll number: ")
    student = find_student(roll)
    if student is not None:
        print(student)
    else:
        print("Student not found.")

def update_student():
    roll = input("Enter roll number: ")
    student = find_student(roll)
    if student is not None:
```

7. Python Source Code (continued)

```
        student["name"] = input("New name: ")
        student["department"] = input("New department: ")
        print("Record updated.")
    else:
        print("Student not found.")

def delete_student():
    roll = input("Enter roll number: ")
    student = find_student(roll)
    if student is not None:
        students.remove(student)
        print("Record deleted.")
    else:
        print("Student not found.")

def display_students():
    if len(students) == 0:
        print("No student records.")
    else:
        for student in students:
            print(student)

def average_marks(student):
    total = 0
    for mark in student["marks"]:
        total = total + mark
    if len(student["marks"]) == 0:
        return 0
    return total / len(student["marks"])

def show_average():
    roll = input("Enter roll number: ")
    student = find_student(roll)
    if student is not None:
        print("Average marks:", average_marks(student))
    else:
        print("Student not found.")

def highest_scorer():
    if len(students) == 0:
        print("No student records.")
        return
    highest = students[0]
    for student in students:
        if average_marks(student) > average_marks(highest):
            highest = student
    print("Highest scorer:", highest["name"])

def list_by_department():
    department = input("Enter department: ")
    found = False
    for student in students:
        if student["department"].lower() == department.lower():
            print(student)
            found = True
    if found == False:
        print("No students in this department.")

def main():
    while True:
        print("\n1 Add  2 Search  3 Update  4 Delete")
        print("5 Display  6 Average  7 Highest scorer")
        print("8 Department list  9 Count  0 Exit")
        choice = input("Choose an option: ")

        if choice == "1":
            add_student()
        elif choice == "2":
            search_student()
        elif choice == "3":
            update_student()
        elif choice == "4":
            delete_student()
        elif choice == "5":
            display_students()
        elif choice == "6":
            show_average()
        elif choice == "7":
            highest_scorer()
        elif choice == "8":
            list_by_department()
        elif choice == "9":
            print("Total students:", len(students))
        elif choice == "0":
            break
        else:
            print("Invalid choice.")

main()
```

8. Test Cases

No.	Test	Expected result
1	Add a new student	Record is added
2	Add duplicate roll number	Warning is shown
3	Search existing roll number	Student details are displayed
4	Search wrong roll number	Not found message
5	Update a record	Record is updated
6	Delete a record	Record is deleted
7	Calculate average	Average is displayed
8	No student records	No-record message
9	List by department	Matching students are displayed

9. Observations

The program stores student records in a list. Each student is stored as a dictionary. It can add, search, update, delete and display records, calculate average marks, find the highest scorer, list students by department and count students.

10. Conclusion

This project explains how Python data structures can be used to organize student records. The algorithms, flowcharts, pseudocode, source code and test cases show the work step by step.